

Processamento de Linguagem Natural

Classificação de Texto com NLTK e Naïve Bayes

Prof. Me. Luis Vinicius

UniAtenas

PLN: o que precisamos saber?

Objetivo

Transformar textos em informações que possam ser analisadas e utilizadas por algoritmos de Machine Learning.

Para isso, precisamos entender:

- 1 Pré-processamento de textos
- 2 Tokenização
- 3 Stopwords
- 4 Stemming
- 5 Vocabulário
- 6 Bag of Words
- 7 Features
- 8 Naive Bayes
- 9 Multinomial Naive Bayes
- 10 Treinamento e teste

1. O que é PLN?

Processamento de Linguagem Natural

PLN é a área que permite que computadores processem, analisem e classifiquem informações presentes em textos e linguagem humana.

Exemplos:

- análise de sentimentos;
- classificação de documentos;
- detecção de spam;
- tradução automática;
- chatbots;
- classificação de nomes.

Exemplo

Texto:

"Este produto é excelente!"

2. Texto → Dados

Um algoritmo de Machine Learning não trabalha diretamente com uma frase como:

"O produto é muito bom!"

Precisamos transformar o texto em uma representação numérica ou estruturada.

Pipeline básico

Texto → Pré-processamento → Representação → Modelo → Classe

Exemplo

"O PRODUTO é MUITO bom!"

↓

"produto muito bom"

↓

vetor numérico

↓

3. Por que pré-processar o texto?

Ideia principal

Pré-processamento reduz ruídos e deixa os textos mais consistentes antes de utilizá-los no modelo.

Um texto pode conter:

- letras maiúsculas e minúsculas;
- HTML;
- pontuação;
- caracteres especiais;
- palavras pouco informativas;
- diferentes formas da mesma palavra.

Exemplo

`<b>Produto EXCELENTE!!!</b>`



Exemplo: limpeza do texto

```
import re

texto = "<b>Produto EXCELENTE!!!</b>"

texto = re.sub("<.*?>", "", texto)
texto = texto.lower()
texto = re.sub(r"[^a-zA-Z\s]", "", texto)

print(texto)
```

Resultado aproximado:

produto excelente

O que aconteceu?

HTML foi removido, letras foram convertidas para minúsculas e caracteres especiais foram retirados.

4. Tokenização

Definição

Tokenização é o processo de dividir um texto em unidades menores chamadas **tokens**.

Exemplo

Texto:

"o produto é excelente"

[o, produto, é, excelente]

Os tokens podem representar:

- palavras;
- números;
- símbolos;
- sentenças.

Exemplo: tokenização com NLTK

```
from nltk.tokenize import word_tokenize  
  
texto = "O produto é excelente."  
  
tokens = word_tokenize(texto)  
  
print(tokens)
```

Resultado:

```
['O', 'produto', 'é', 'excelente', '.']
```

Por que tokenizar?

Porque operações posteriores, como remoção de stopwords, stemming e construção de vocabulário, normalmente trabalham sobre tokens.

5. Stopwords

Definição

Stopwords são palavras muito frequentes que geralmente possuem pouco poder discriminativo em determinada tarefa.

Exemplos em inglês:

the, a, an, is, of, to, and

Exemplo

"the product is very good"

↓

"product very good"

Importante:

A remoção de stopwords é uma etapa de pré-processamento. Ela não significa que essas palavras sejam "erradas".

Exemplo: removendo stopwords

```
from nltk.corpus import stopwords

tokens = ["the", "product", "is", "very", "good"]

stop = set(stopwords.words("english"))

resultado = [
    w for w in tokens
    if w not in stop
]

print(resultado)
```

Resultado:

```
['product', 'very', 'good']
```

6. Stemming

Definição

Stemming reduz palavras para uma forma simplificada, normalmente removendo partes do final da palavra.

Exemplo

Palavras relacionadas:

connect

connected

connecting

podem ser reduzidas para uma forma semelhante:

Exemplo: Stemming com NLTK

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()

palavras = [
    "connect",
    "connected",
    "connecting"
]

for palavra in palavras:
    print(stemmer.stem(palavra))
```

Possível resultado:

```
connect
connect
connect
```

7. Vocabulário

Definição

Vocabulário é o conjunto de palavras distintas consideradas em um conjunto de textos.

Exemplo

Textos:

"good product"

"good service"

Vocabulário:

$\{\text{good, product, service}\}$

O vocabulário será importante para transformar textos em representações estruturadas.

Exemplo: criando um vocabulário

```
textos = [  
    ["good", "product"],  
    ["good", "service"]  
]  
  
vocab = set()  
  
for texto in textos:  
    vocab.update(texto)  
  
print(vocab)
```

Resultado:

```
{'good', 'product', 'service'}
```

8. Bag of Words – BoW

Definição

Bag of Words representa um texto por meio da presença, frequência ou contagem das palavras de um vocabulário.

Suponha o vocabulário:

[good, bad, product]

Texto:

"good product good"

Representação por contagem:

[2, 0, 1]

- good: 2 ocorrências
- bad: 0 ocorrências

Exemplo: Bag of Words

```
from sklearn.feature_extraction.text import CountVectorizer

textos = [
    "good product",
    "bad product",
    "good service"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(textos)

print(vectorizer.get_feature_names_out())
print(X.toarray())
```

Uma possível representação:

['bad' 'good' 'product' 'service']

[[0 1 1 0]

[1 0 1 0]]

9. Por que o BoW é importante?

O algoritmo de Machine Learning precisa de uma representação que possa ser processada matematicamente.

Transformação

Texto $\longrightarrow$ BoW $\longrightarrow$ Vetor numérico

Exemplo

"good product"



[0, 1, 1, 0]

O vetor pode ser utilizado como entrada para um classificador.

Atenção

BoW não classifica o texto.

10. Naive Bayes

Ideia

Naive Bayes calcula a probabilidade de uma classe considerando as características observadas no exemplo.

Para análise de sentimentos:

$$P(\text{positivo} \mid \text{palavras})$$

e

$$P(\text{negativo} \mid \text{palavras})$$

O modelo compara essas probabilidades.

Exemplo

Texto:

"excellent product"

11. A ideia matemática do Naive Bayes

Para uma classe C e palavras observadas $w_1, \dots, w_n$:

$$P(C \mid w_1, \dots, w_n) \propto P(C) \prod_{i=1}^n P(w_i \mid C)$$

Onde:

- $P(C)$ = probabilidade da classe;
- $P(w_i \mid C)$ = probabilidade da palavra aparecer naquela classe;
- $P(C \mid \textit{palavras})$ = probabilidade da classe dado o texto.

Decisão

A classe com maior probabilidade é escolhida.

Exemplo simples de Naive Bayes

Suponha:

$$P(\textit{positive}) = 0.6$$

$$P(\textit{negative}) = 0.4$$

Para a palavra *excellent*:

$$P(\textit{excellent} \mid \textit{positive}) = 0.8$$

$$P(\textit{excellent} \mid \textit{negative}) = 0.1$$

Então:

$$\textit{positive} = 0.6 \times 0.8 = 0.48$$

12. Por que Multinomial Naive Bayes?

Ideia

O Multinomial Naive Bayes é adequado quando as características representam contagens ou frequências de eventos.

No BoW podemos ter:

[good = 3, bad = 0, product = 2]

Essas contagens são naturalmente utilizadas pelo modelo.

Exemplo

Texto:

"good good good product product"

BoW:

[3, 0, 2]

Exemplo: MultinomialNB

```
from sklearn.naive_bayes import MultinomialNB

modelo = MultinomialNB()

modelo.fit(X_train, y_train)

previsoes = modelo.predict(X_test)
```

Pipeline:

Texto → BoW → MultinomialNB → Classe

13. O que é uma Feature?

Definição

Feature é uma característica observável utilizada pelo modelo para realizar uma classificação.

Para classificar nomes, podemos utilizar:

- primeira letra;
- última letra;
- duas últimas letras;
- presença de determinada letra;
- quantidade de determinada letra;
- tamanho do nome.

Nome

Jacqueline

`first_letter = j`

Exemplo: extraindo Features

```
def extract_features(nome):  
    nome = nome.lower()  
  
    return {  
        "first_letter": nome[0],  
        "suffix_1": nome[-1],  
        "suffix_2": nome[-2:],  
        "length": len(nome)  
    }  
  
print(extract_features("Jacqueline"))
```

Resultado:

```
{  
    'first_letter': 'j',  
    'suffix_1': 'e',  
    'suffix_2': 'ne',  
    'length': 10  
}
```

14. Por que criar Features?

O nome completo:

Jacqueline

é transformado em características:

```
{first_letter:j, suffix_1:e, suffix_2:ne}
```

Função da função `extract_gender_features()`

Ela recebe um nome e extrai características relevantes que serão utilizadas pelo classificador.

Importante

A função de extração de features não é o classificador.

Ela prepara as informações que serão utilizadas pelo classificador.

15. Por que o sufixo pode ajudar?

Considere apenas a última letra:

`suffix_1 = a`

Isso fornece uma informação limitada.

Com duas letras:

`suffix_2 = na`

temos uma informação mais específica.

Exemplo

Considere nomes terminados em:

`-a`

Muitos nomes podem possuir essa terminação.

Mas:

Exemplo: suffix_1 × suffix_2

```
nome = "Juliana"

suffix_1 = nome[-1]
suffix_2 = nome[-2:]

print(suffix_1)
print(suffix_2)
```

Resultado:

a

na

Informação

suffix_1 informa apenas a última letra.

suffix_2 captura uma sequência de duas letras.

16. Mais Features $\neq$ necessariamente melhor

Adicionar features pode:

- acrescentar informação útil;
- não alterar o desempenho;
- acrescentar informação pouco relevante;
- introduzir ruído;
- aumentar a complexidade do modelo.

Conclusão

A quantidade de features não determina sozinha a qualidade do classificador.
O importante é a **informatividade** das features.

Exemplo

76% com 4 features

76% com 10 features

78% com 11 features

Mais features não produziram automaticamente melhor resultado.

Exemplo: várias Features

```
def features(nome):  
    nome = nome.lower()  
  
    return {  
        "first": nome[0],  
        "last": nome[-1],  
        "suffix2": nome[-2:],  
        "has_a": "a" in nome,  
        "count_a": nome.count("a")  
    }
```

Cada nova feature adiciona informação ao exemplo.

Pergunta importante

A informação adicionada realmente ajuda a separar as classes?

17. Treinamento × Teste

Conjunto de treinamento

Dados utilizados para que o modelo aprenda os padrões.

Conjunto de teste

Dados separados para verificar o desempenho do modelo em exemplos que ele não utilizou durante o treinamento.

Exemplo:

50.000 avaliações

80% → *treinamento*

20% → *teste*

Exemplo: divisão dos dados

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Resultado:

80% $\rightarrow X_{train}, y_{train}$

20% $\rightarrow X_{test}, y_{test}$

Por que não avaliar somente no treinamento?

Porque o modelo pode simplesmente ter aprendido muito bem os exemplos que já viu, sem conseguir generalizar para novos dados.

18. Generalização

Ideia

Um bom modelo deve aprender padrões gerais dos dados, e não simplesmente memorizar os exemplos de treinamento.

Exemplo

Treinamento:

good, excellent, wonderful

Teste:

amazing, fantastic

Queremos saber se o modelo consegue reconhecer o padrão positivo mesmo diante de palavras que não aparecem exatamente da mesma forma no treinamento.

19. Acurácia

Definição

Acurácia é a proporção de previsões corretas em relação ao total de previsões.

$$\text{Acurácia} = \frac{\text{previsões corretas}}{\text{total de previsões}}$$

Exemplo

100 avaliações no conjunto de teste.

O modelo acertou 78.

$$\text{Acurácia} = \frac{78}{100} = 78\%$$

Exemplo: calculando acurácia

```
from sklearn.metrics import accuracy_score

y_real = ["positive", "negative", "positive"]
y_pred = ["positive", "positive", "positive"]

acuracia = accuracy_score(y_real, y_pred)

print(acuracia)
```

Resultado:

0.6667

ou aproximadamente:

66,67%

20. Outras métricas

Além da acurácia, podemos utilizar:

- precisão (*precision*);
- revocação (*recall*);
- F1-score;
- matriz de confusão.

F1-score

Combina precisão e revocação em uma única medida:

$$F1 = 2 \frac{Precision \cdot Recall}{Precision + Recall}$$

Quando é útil?

Quando não queremos analisar somente a quantidade total de acertos, mas também como os erros estão distribuídos entre as classes.

Exemplo: métricas com Python

```
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score  
)  
  
print(accuracy_score(y_test, y_pred))  
  
print(precision_score(  
    y_test, y_pred,  
    pos_label="positive"  
))  
  
print(recall_score(  
    y_test, y_pred,  
    pos_label="positive"  
))  
  
print(f1_score(  
    y_test, y_pred,  
    pos_label="positive"
```

21. Vocabulário do NLTK

Considere:

```
english_vocab = {  
    w.lower()  
    for w in words.words()  
}
```

O que representa?

Um conjunto de palavras presentes no vocabulário fornecido pelo NLTK, convertidas para letras minúsculas.

Ideia

O NLTK possui uma lista de palavras em inglês.

$$\text{english_vocab} = \{\text{palavras conhecidas pelo recurso}\}$$

22. text_vocab e isalpha()

Código:

```
text_vocab = {  
    w.lower()  
    for w in text_tokenized  
    if w.isalpha()  
}
```

Por que usar `isalpha()`?

Para seleccionar tokens formados somente por letras.

Exemplo

```
["hello", ",", "world", "123", "!"]
```

Após `isalpha()`:

```
["hello", "world"]
```

Exemplo: isalpha()

```
tokens = [  
    "hello",  
    ",",  
    "world",  
    "123",  
    "!"  
]  
  
for w in tokens:  
    if w.isalpha():  
        print(w)
```

Saída:

hello
world

Atenção

`isalpha()` verifica se o token é composto por caracteres alfabéticos. Não verifica se a palavra existe no dicionário.

23. difference()

Considere:

$$A = \{\text{google, puzzles, computer}\}$$

e:

$$B = \{\text{computer, language}\}$$

Então:

$$A - B = \{\text{google, puzzles}\}$$

Em Python:

```
A.difference(B)
```

No problema

```
text_vocab.difference(english_vocab)
```

significa:

Exemplo completo: palavras incomuns

```
english_vocab = {  
    w.lower()  
    for w in words.words()  
}  
  
text_vocab = {  
    w.lower()  
    for w in text_tokenized  
    if w.isalpha()  
}  
  
unusual = text_vocab.difference(  
    english_vocab  
)  
  
print(unusual)
```

Se:

```
{'google', 'puzzles'}
```

então essas palavras aparecem no texto, mas não estão no vocabulário utilizado pelo NLTK.

24. Por que uma palavra conhecida pode ser “incomum”?

O resultado não significa necessariamente:

google não é uma palavra

ou:

puzzles não existe em inglês

Significa apenas:

Interpretação correta

A palavra está no texto, mas não foi encontrada no conjunto de palavras utilizado como referência.

Possíveis motivos:

- vocabulário limitado;
- versão diferente do recurso;
- nomes próprios;

25. Resumo: pipeline de análise de sentimentos

Avaliações



Limpeza



Tokenização



Stopwords / Stemming

26. Resumo: classificação de nomes

Nome



`extract_gender_features()`



`{first, last, suffix2, has(a), count(a)}`



NaiveBayesClassifier



masculino / feminino